

Instructions for preparing the solution script:

- Write your name, ID#, and Section number clearly in the very front page.
 - Your submission must be a handwritten version.
 - Start working right away. No late submission will be accepted for the assignment.
-

Assignment Task Description [10 Marks]:

You have to study the intermediate code generation from Section 6.9 of the reference [textbook](#).

Problem Statement

The front-end analysis of a program involves parsing the source code according to a grammar and attaching semantic rules to produce intermediate code. In this assignment, you are given a small code snippet that includes a recursive function, arithmetic operations, and a function call. Also, an appropriate grammar capable of parsing this program and defining semantic rules that generate three-address code (TAC) during parsing. Using the grammar and semantic rules, construct the parse tree for the program and then generate the corresponding TAC. This exercise is intended to help you understand how syntax and semantics work together to produce intermediate code representations in a compiler.

Syntax Directed Translation (SDT)

→ Go to the next page.

Code Block for Translation

```
int fibonacci(int n) {  
    if (n <= 1) return n;  
    else return fibonacci(n-1) + fibonacci(n - 2);  
}
```

Expected Answers

1. [5 marks] **Annotated Parse Tree:** Draw an annotated parse tree for the input program, considering the SDT.
 2. [5 marks] **Three-Address Code (TAC):** Generate TAC step by step using the SDT.
-

```

-----
Helper routines (Pseudocode)
-----
newtemp()  -> t1, t2, t3, ...
newlabel() -> L1, L2, L3, ...

func mkNode(name : string, children : list<any>, ann : map<string,
any>) -> Node
    n          ← new Node
    n.name     ← name
    n.children ← children
    n.ann      ← ann
    return n

func gen(params[]) {
    str = "";
    for (param in params)
        str = str || param;
    return str;}
-----
Attributes
-----
For every nonterminal X:
    X.node : annotated parse-tree node

For types:
    T.type : string

For expressions:
    E.place, Term.place, Factor.place, Call.place : location
(id/num/temp) holding value

For boolean:
    B.truelabel, B.falselabel : labels to branch to

Markers (for correct emission order):
    MF.funcName : string
    M.label : label
    N.falseLabel, N.endLabel : label

For terminals:
    id.lexeme, num.lexeme

```

```

-----
Grammar + Semantic Rules/Actions
-----

P → F
    {P.node = mkNode("P", [F.node], {})}

F → T id "(" Param ")" MF Block
    {gen("endfunc")
     F.node = mkNode("F",
                     [T.node, id.lexeme, Param.node, Block.node],
                     {"name": id.lexeme, "type": T.type})}

MF → ε
    //MF.funcName = id.lexeme is set by parent rule (F → T id "("
Param ")" MF Block) before expanding MF
    {gen("func " + MF.funcName)
     MF.node = mkNode("MF", [], {"func": MF.funcName})}

T → "int"
    {T.type = "int"
     T.node = mkNode("T", ["int"], {"type": "int"})}

Param → T id
    {Param.node = mkNode("Param", [T.node, id.lexeme],
                        {"name": id.lexeme, "type": T.type})}

Block → "{" S "}"
    {Block.node = mkNode("Block", [S.node], {})}

S → IfStmt
    {S.node = mkNode("S", [IfStmt.node], {})}

IfStmt → "if" "(" B ")" M Stmt1 N "else" Stmt2
    {Ltrue  = newlabel()
     Lfalse = newlabel()
     Lend   = newlabel()
     B.truelabel  = Ltrue
     B.falseLabel = Lfalse
     M.label      = Ltrue
     N.falseLabel = Lfalse
     N.endLabel   = Lend

```

```

// after Stmt2 completes:
gen(Lend + ":")
IfStmt.node = mkNode("IfStmt",
                    [B.node, Stmt1.node, Stmt2.node],
                    {"Ltrue":Ltrue,"Lfalse":Lfalse,"Lend":Lend})}

```

M → ϵ

```

{gen(M.label + ":")
 M.node = mkNode("M", [], {"label": M.label})}

```

N → ϵ

```

{gen("goto " + N.endLabel)
 gen(N.falseLabel + ":")
 N.node = mkNode("N", [],
                {"false": N.falseLabel, "end": N.endLabel})}

```

Stmt → **Return** ";"

```

{Stmt.node = mkNode("Stmt", [Return.node], {})}

```

Return → **"return"** **E**

```

{gen("return " + E.place)
 Return.node = mkNode("Return", [E.node],
                    {"returnPlace": E.place})}

```

B → **E₁ Relop E₂**

```

{gen("if " + E1.place + " " + Relop.op + " " + E2.place
    + " goto " + B.truelabel)
 gen("goto " + B.falselabel)
 B.node = mkNode("B", [E1.node, Relop.node, E2.node],
                {"true":B.truelabel,"false":B.falselabel,
                 "lhs":E1.place,"rhs":E2.place})}

```

Relop → **"<="**

```

{Relop.op = "<="
 Relop.node = mkNode("Relop", ["<="], {"op":"<="})}

```

E → **E₁ "-" Term**

```

{t = newtemp()
 E.place = t
 gen(t + " = " + E1.place + " - " + Term.place)
 E.node = mkNode("E-", [E1.node, Term.node], {"place":t})}

```

E → E₁ "+" Term

```
{t = newtemp()
  E.place = t
  gen(t + " = " + E1.place + " + " + Term.place)
  E.node = mkNode("E+", [E1.node, Term.node], {"place":t})}
```

E → Term

```
{E.place = Term.place
  E.node = mkNode("E", [Term.node], {"place":E.place})}
```

Term → Factor

```
{Term.place = Factor.place
  Term.node = mkNode("Term",
    [Factor.node], {"place":Term.place})}
```

Factor → id

```
{Factor.place = id.lexeme
  Factor.node = mkNode("id",
    [id.lexeme], {"place":Factor.place})}
```

Factor → num

```
{Factor.place = num.lexeme
  Factor.node = mkNode("num", [
    num.lexeme], {"place":Factor.place})}
```

Factor → Call

```
{Factor.place = Call.place
  Factor.node = mkNode("FactorCall",
    [Call.node], {"place":Factor.place})}
```

Factor → "(" E ")"

```
{Factor.place = E.place
  Factor.node = mkNode("Paren",
    [E.node], {"place":Factor.place})}
```

Call → id "(" E ")"

```
{t = newtemp()
  Call.place = t
  gen("param " + E.place)
  gen(t + " = call " + id.lexeme + ", 1")
  Call.node = mkNode("Call", [id.lexeme, E.node],
    {"fn":id.lexeme,"argc":1,"place":t})}
```